

XXVI SIMPÓSIO BRASILEIRO DE CIBERSEGURANÇA · SBSEG 2026

1 a 4 de setembro de 2026 · Armação dos Búzios, RJ

Security-First Evaluation of Text-to-Terraform: Benchmarking LLMs and SLMs for Secure IaC Generation



Francis Vargas, Rodrigo Brandão Mansilha, Diego Kreutz

AI Horizon Labs · Universidade Federal do Pampa (UNIPAMPA)

Programa de Pós-Graduação em Engenharia de Software (PPGES)



O problema

laC acelera a implantação, mas também escala configurações inseguras e exposições que podem permanecer meses sem detecção.

23%

dos incidentes de nuvem vêm de recursos mal configurados



Threat #1

**Top Threat #1 —
Misconfiguration**



40%

De código vulnerável ainda é gerado por IA em cenários sensíveis



DESAFIOS TÉCNICOS

Sintaxe válida não é sinônimo de seguro

Terraform pode aceitar configurações que violam controles essenciais de segurança.

```
# bucket_policy.tf - trecho do estudo
resource "aws_s3_bucket_public_access_block" "example" {
  ✗ block_public_acls      = false
  ✗ block_public_policy    = false
  ✗ ignore_public_acls     = false
  ✗ restrict_public_buckets = false
}
```

terraform validate

✓ **PASS**

security scanner · checkov

✗ **FAIL**

WizardCoder-33B · nível de segurança L3

77,8%

✓ terraform válido

0%

✗ conformidade Checkov

Mensagem-chave: o código pode passar na validação do Terraform e, ainda assim, chegar ao deployment com falhas de segurança, validade sintática e conformidade de segurança são propriedades independentes.

DESAFIOS TÉCNICOS

O que já existe e o que falta

Trabalho	Modelos	Scanner(s)	Lacuna que permanece
IaC-Eval	LLM+SLM	OPA	Sem verificação de segurança
DPIaC-Eval	LLM	Checkov	Só 8,4% de conformidade após correção
Multi-IaC-Eval	LLM+SLM	CFN-Lint	Sem scanner de segurança
TerraFormer	LLM	Checkov	SLMs fora do foco de segurança
Este trabalho	LLM+SLM	Checkov + Trivy	Laboratório mais extenso de experimentação

Nenhum trabalho anterior combina múltiplos modelos, múltiplos scanners e correção seletiva por feedback.

Quatro perguntas de pesquisa

RQ1 · Validade e segurança

Validade sintática e conformidade de segurança são correlacionadas em LLMs e SLMs?

RQ2 · Especificidade do prompt

Aumentar a especificidade do prompt melhora substancialmente a conformidade de segurança?

RQ3 · Autocorreção por scanner

Modelos conseguem se autocorrigir quando recebem feedback bruto de Checkov e Trivy?

RQ4 · Evolução longitudinal

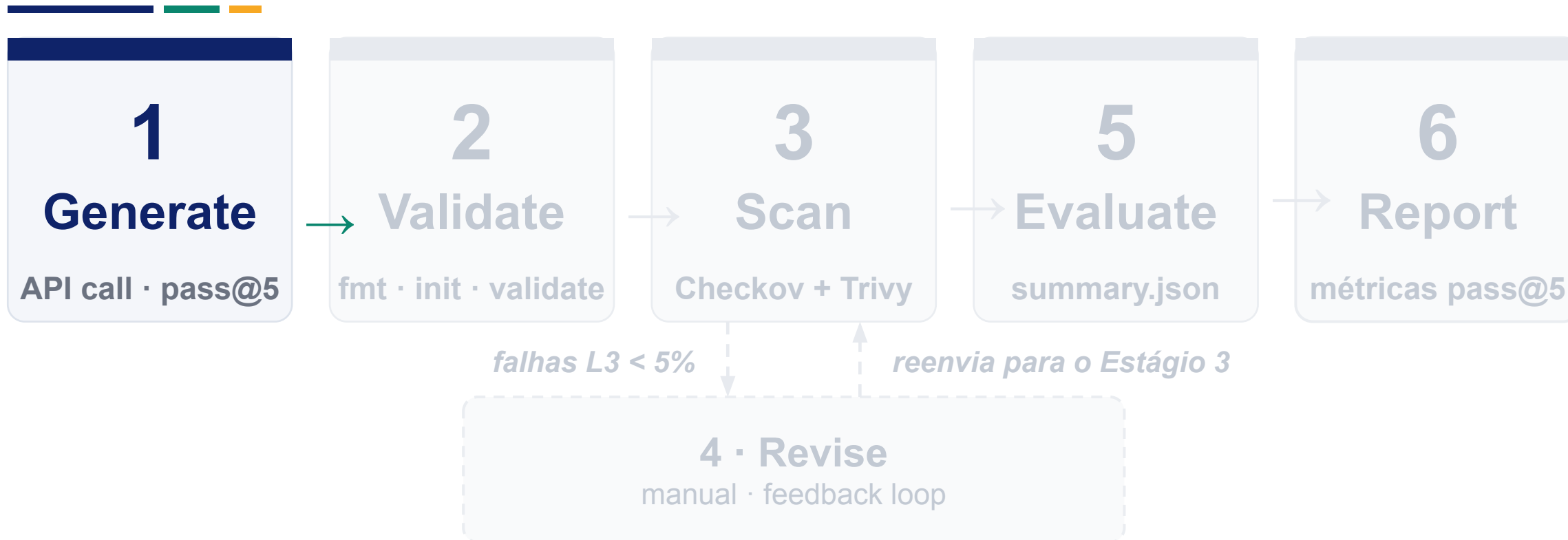
As SLMs evoluíram em segurança entre o IaC-Eval (2024) e esta avaliação (2026)?

01

Metodologia

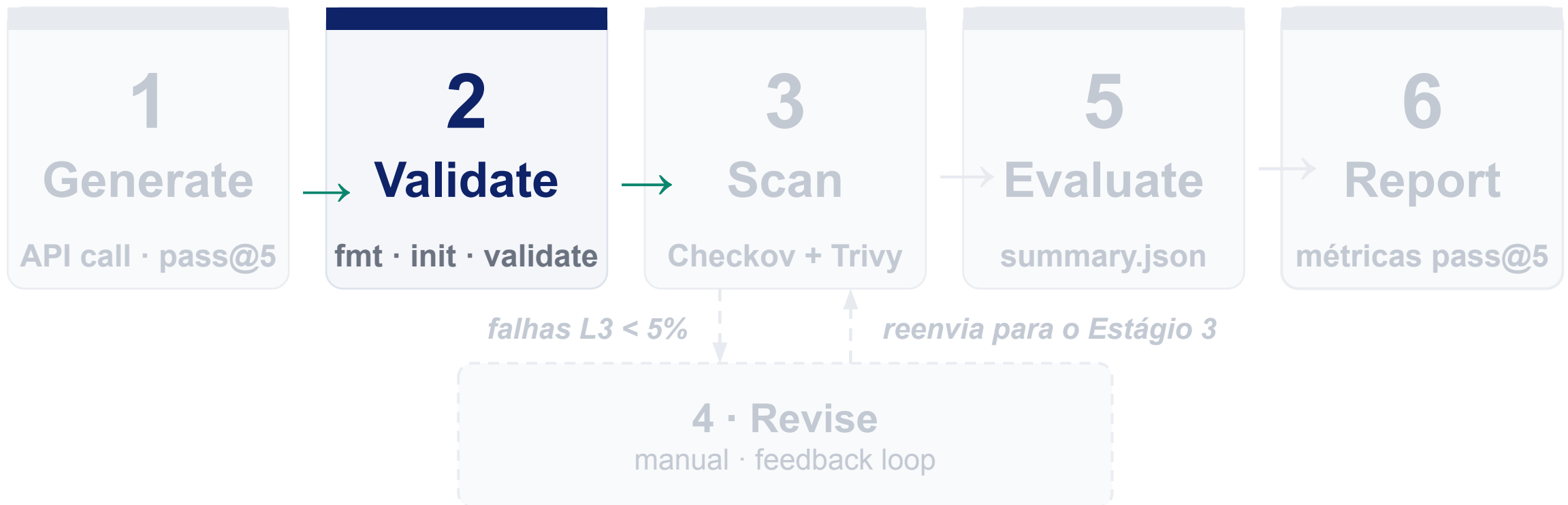
Como desenhamos um benchmark security-first

Pipeline do benchmark



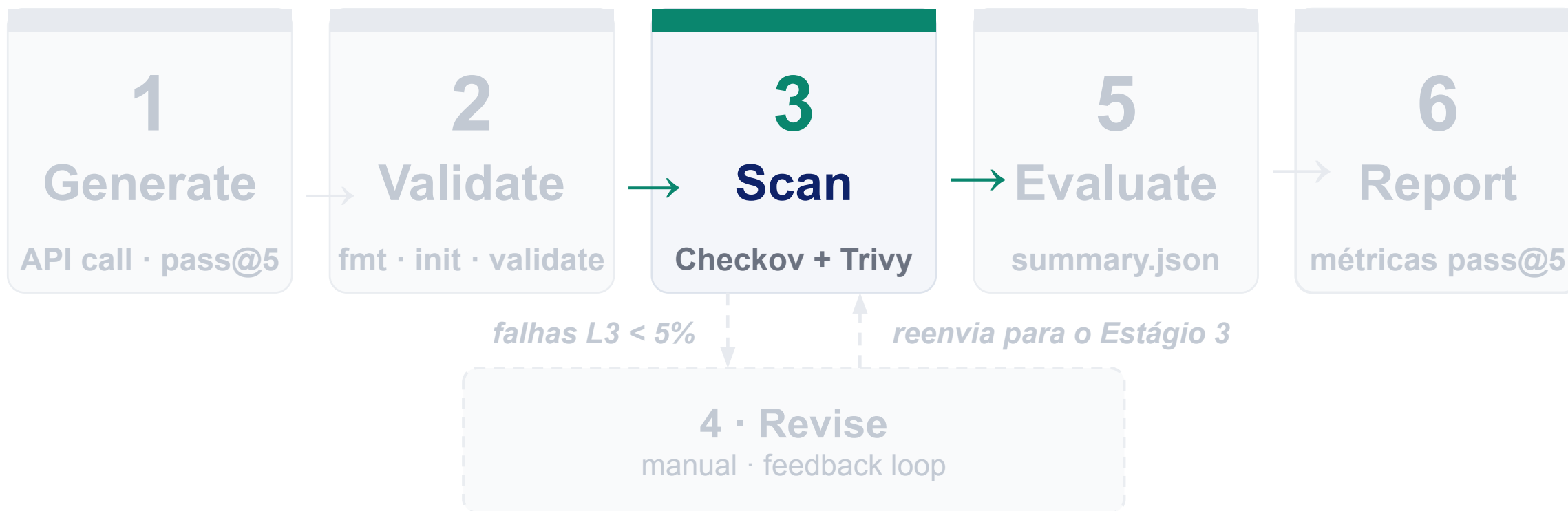
Estágio 1 · Generate — cada um dos 7 modelos gera Terraform via API, 5 vezes por cenário (pass@5), 3.570 avaliações no total.

Pipeline do benchmark



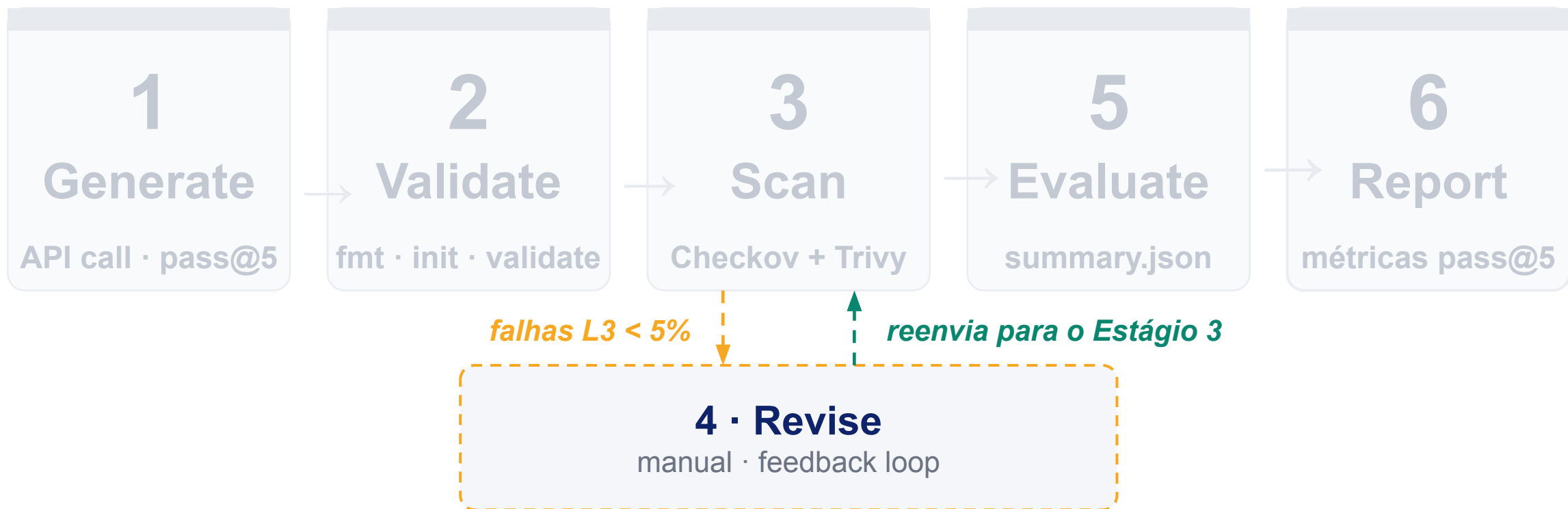
Estágio 2 · Validate — terraform *fmt*, *init* e *validate* confirmam apenas a corretude sintática do código; nada aqui avalia segurança.

Pipeline do benchmark



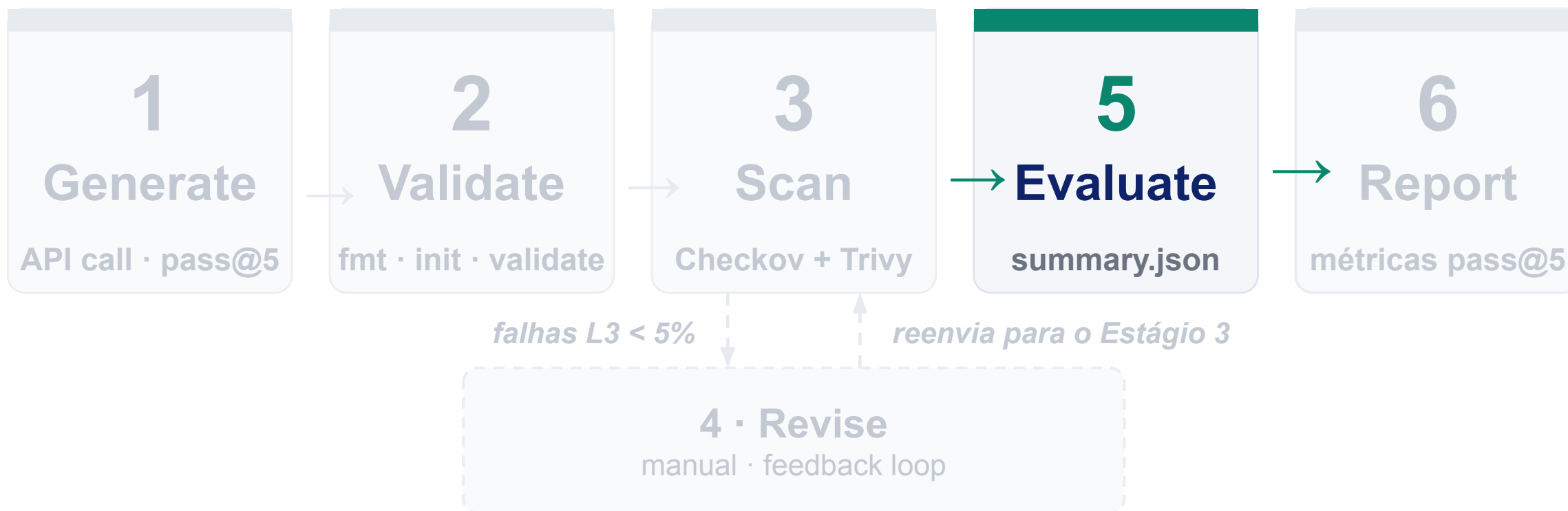
Estágio 3 · Scan — Checkov (conformidade) e Trivy (CVEs) analisam o mesmo plano; um único scanner de segurança não basta.

Pipeline do benchmark



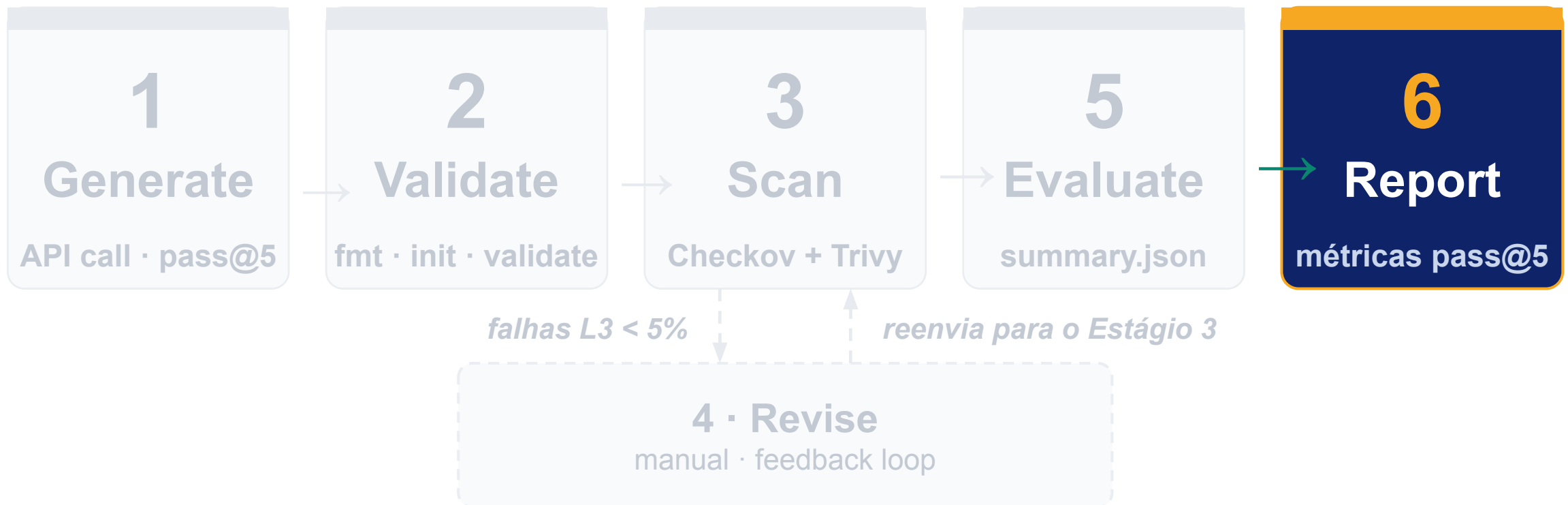
Estágio 4 · Revise — acionado manualmente apenas quando a conformidade L3 fica abaixo de 5%; o código revisado reentra no Estágio 3.

Pipeline do benchmark



Estágio 5 · Evaluate — os achados dos dois scanners são agregados por cenário e nível de segurança em *summary.json*.

Pipeline do benchmark



Estágio 6 · Report — métricas finais de pass@5 por modelo, nível e scanner: o ponto de chegada do benchmark.

Modelos, cenários e níveis de segurança

- 7 modelos: 3 LLMs fechados (GPT-5.4, Gemini 2.5 Pro, Claude Opus 4) + 4 SLMs abertas.
- 17 cenários AWS (S3, KMS, VPC), por consenso unânime de 6 LLMs avaliadores.
- Zero-shot / few-shot × 3 níveis de segurança (L1, L2, L3 detalhado).
- 5 repetições por célula, 3.570 avaliações no total.

02

Avaliação experimental

O que 3.570 avaliações revelam sobre segurança em IaC

02 · AVALIAÇÃO EXPERIMENTAL

Configuração experimental

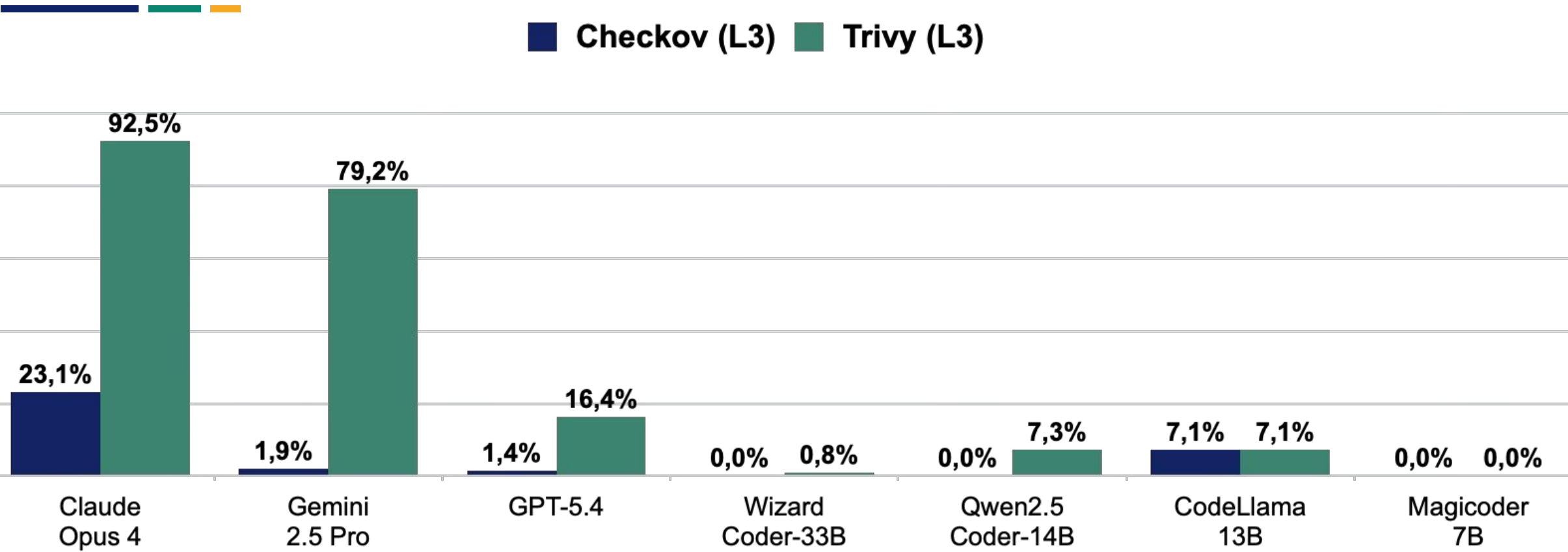
Artefato →



Ferramenta	Versão	Papel
Terraform	1.7.5	fmt, init, validate e plan
AWS Provider	5.100.0	Resolvido em terraform init
Checkov	3.2.527	Scanner de conformidade CKV_AWS_*
Trivy	0.69.3	Scanner de vulnerabilidades AVD-AWS-*
GitLab CE/EE	19.0.0-pre	Orquestração CI/CD

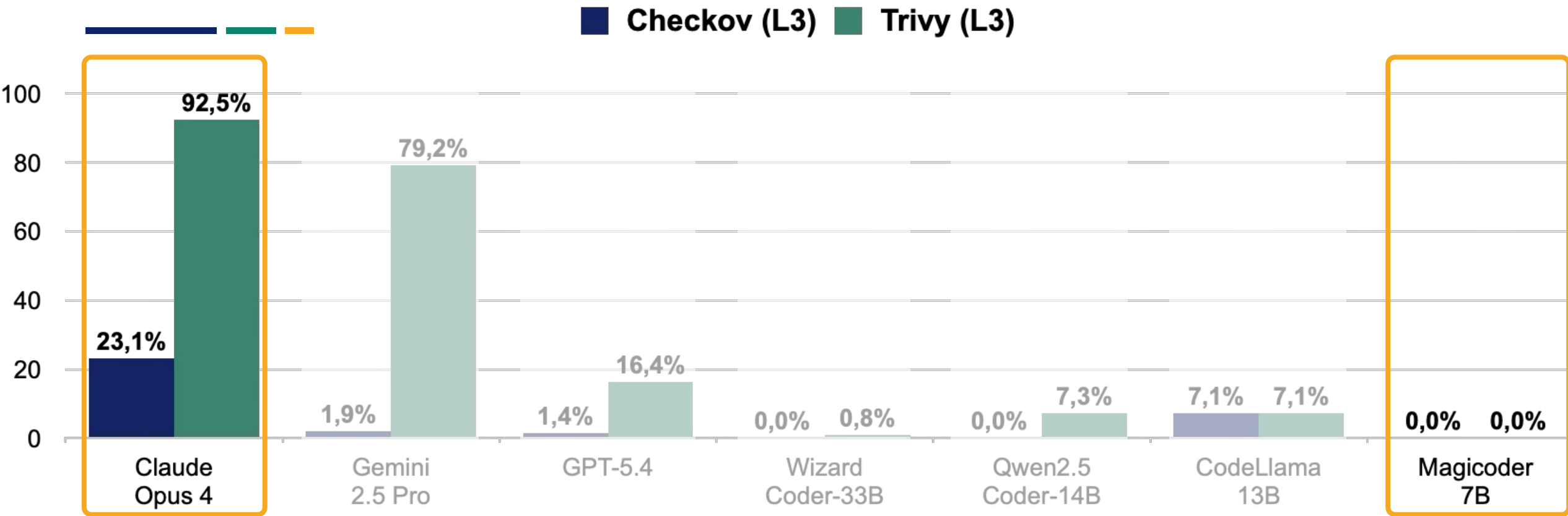
02 · AVALIAÇÃO EXPERIMENTAL

Resultados



02 · AVALIAÇÃO EXPERIMENTAL

Resultados

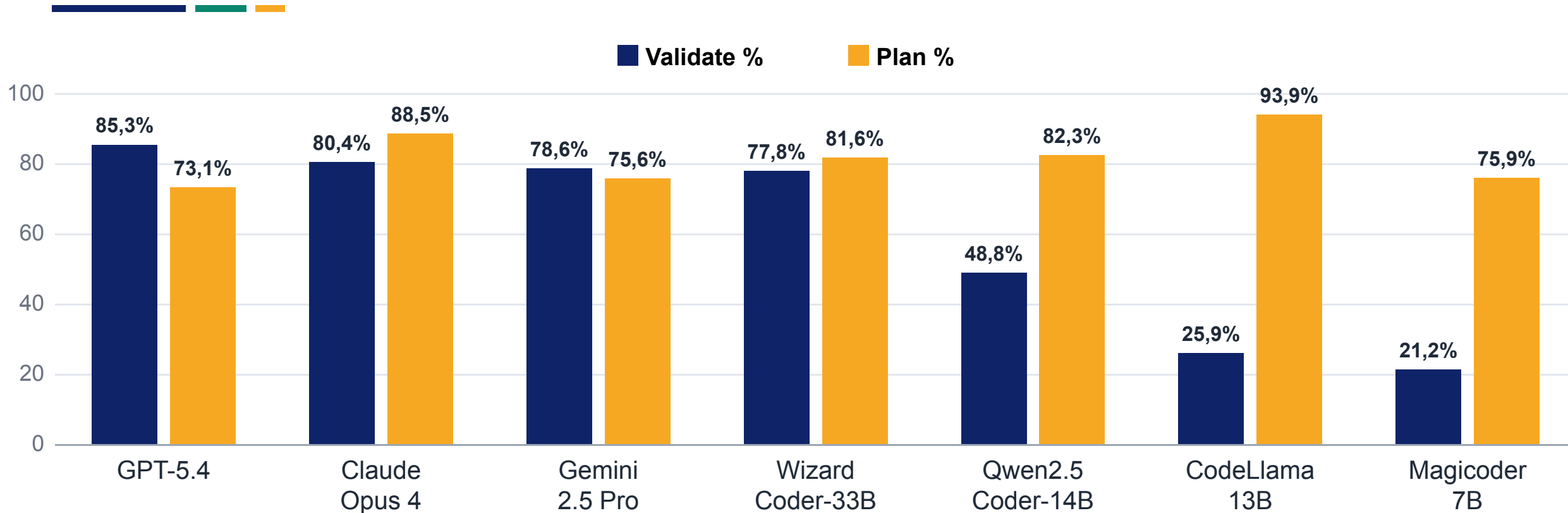


Claude Opus 4
92,5% Trivy · **23,1%** Checkov

Magicoder-7B
0,0% Trivy · **0,0%** Checkov

02 · AVALIAÇÃO EXPERIMENTAL

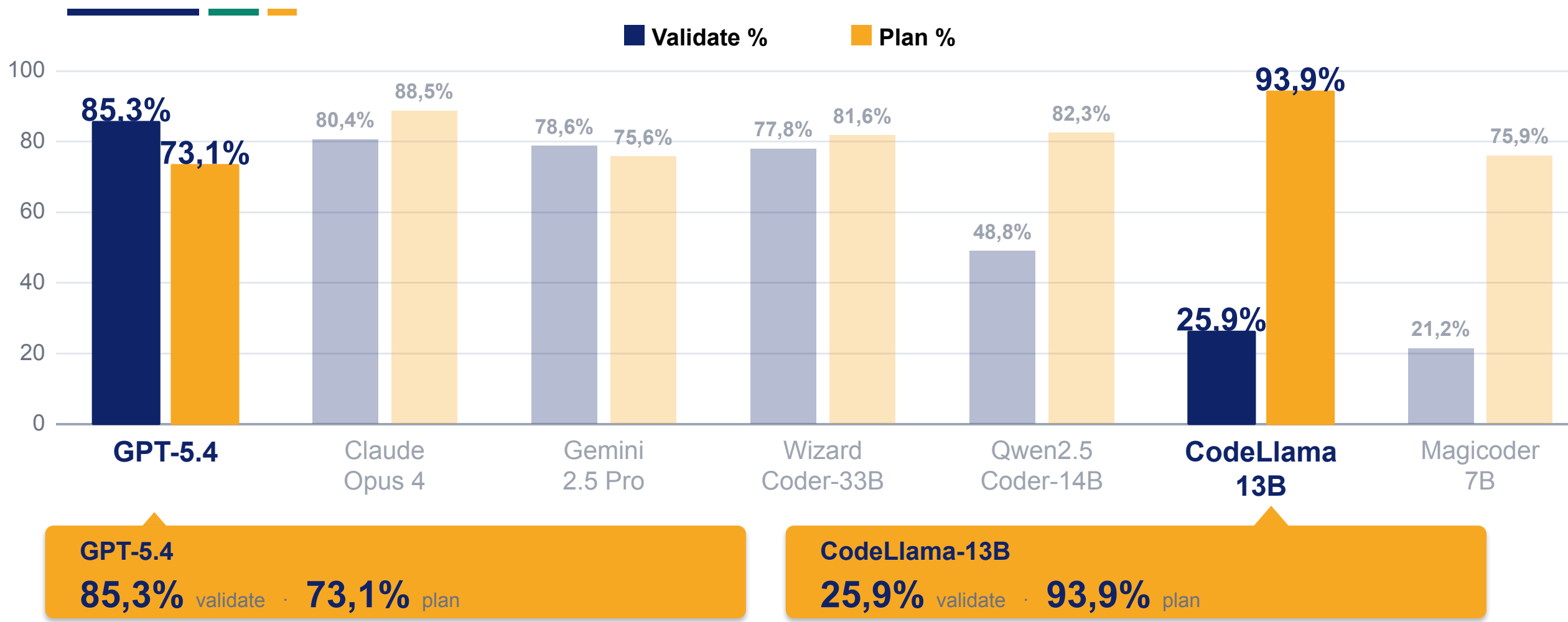
Validade Sintática × Executabilidade do Plan



Achado-chave: GPT-5.4 lidera em validate (85,3%), mas tem o menor plan% entre LLMs (73,1%); CodeLlama-13B é o oposto (25,9% validate, 93,9% plan).

02 · AVALIAÇÃO EXPERIMENTAL

Validade Sintática × Executabilidade do Plan



RQ2 · Efeito da especificidade do prompt (RQ2)

Modelo	Tipo	Checkov L1 → L2 → L3	Trivy L1 → L2 → L3
GPT-5.4	LLM	0 → 0 → 2 (1,4%)	2 → 51 → 24 (16,4%)
Claude Opus 4	LLM	0 → 4 → 31 (23,1%)	4 → 102 → 124 (92,5%)
Gemini 2.5 Pro	LLM	0 → 0 → 2 (1,9%)	5 → 78 → 84 (79,2%)
WizardCoder-33B	SLM	0 → 0 → 0 (0,0%)	0 → 1 → 1 (0,8%)
Qwen2.5-Coder-14B	SLM	0 → 0 → 0 (0,0%)	0 → 2 → 6 (7,3%)
CodeLlama-13B	SLM	1 → 0 → 1 (7,1%)	0 → 1 → 1 (7,1%)
Magocoder-7B	SLM	0 → 0 → 0 (0,0%)	0 → 0 → 0 (0,0%)

RQ2 · Efeito da especificidade do prompt (RQ2)

Modelo	Tipo	Checkov L1 → L2 → L3	Trivy L1 → L2 → L3
GPT-5.4	LLM	0 → 0 → 2 (1,4%)	2 → 51 → 24 (16,4%)
Claude Opus 4	LLM	0 → 4 → 31 (23,1%)	4 → 102 → 124 (92,5%)
Gemini 2.5 Pro	LLM	0 → 2 (1,9%)	5 → 78 → 84 (79,2%)
WizardCoder-33B	SLM		
Qwen2.5-Coder-14B	SLM		
CodeLlama-13B	SLM	1 → 0 → 1 (7,1%)	0 → 1 → 1 (7,1%)
Magocoder-7B	SLM	0 → 0 → 0 (0,0%)	0 → 0 → 0 (0,0%)

- Checkov: 0 → 4 → 31 (23,1%)
- Trivy: 4 → 102 → 124 (92,5%)
- Maior salto do L1 ao L3 entre os 7 modelos

RQ3 · Correção por feedback de scanner

Modelo	Trivy R1 → Revisão	Checkov R1 → Revisão	Δ Trivy
GPT-5.4	17,7% → 61,4%	0,5% → 2,8%	+43,7pp
WizardCoder-33B	0,5% → 42,1%	0,0% → 0,0%	+41,6pp
CodeLlama-13B	3,8% → 26,9%	3,0% → 3,8%	+23,1pp
Claude Opus 4	56,8% → 49,7%*	8,5% → 20,8%	+12,3pp Chk
Gemini 2.5 Pro	42,9% → 39,0%*	0,5% → 10,0%	+9,5pp Chk
Magocoder-7B	0,9% → 13,2%	0,0% → 2,6%	+12,3pp

RQ3 · Correção por feedback de scanner

Modelo	Trivy R1 → Revisão	Checkov R1 → Revisão	Δ Trivy
GPT-5.4	17,7% → 61,4%	0,5% → 2,8%	+43,7pp
WizardCoder-33B	0,5% → 42,1%	0,0% → 0,0%	+41,6pp
CodeLlama-13B			+23,1pp
Claude Opus 4			+12,3pp Chk
Gemini 2.5 Pro	42,9% → 39,0%*	0,5% → 10,0%	+9,5pp Chk
Magocoder-7B	0,9% → 13,2%	0,0% → 2,6%	+12,3pp

- 17,7% → 61,4% em Trivy após revisão
- Maior ganho entre os 7 modelos
- Checkov quase não muda (0,5%→2,8%)

RQ3 · Correção por feedback de scanner

Modelo	Trivy R1 → Revisão	Checkov R1 → Revisão	Δ Trivy
GPT-5.4	17,7% → 61,4%	0,5% → 2,8%	+43,7pp
WizardCoder-33B	0,5% → 42,1%	0,0% → 0,0%	+41,6pp
CodeLlama-13B	3,8% → 26,9%	3,0% → 3,8%	+23,1pp
Claude Opus 4	56,8% → 49,7%*	8,5% → 20,8%	+12,3pp Chk
Gemini 2.5 Pro	42,9% → 39,0%*	0,5% → 10,0%	+9,5pp Chk
Magocoder-7B	0,9% → 13,2%	0,0% → 2,6%	+12,3pp

Resultados em três números

92,5%

Trivy (Claude Opus 4)

Maior taxa de conformidade de segurança do benchmark.

0%

Checkov (WizardCoder)

77,8% validate, porém nenhuma geração é compliant.

8,7×

Validade · 2024 → 2026

WizardCoder-33B: de 8,9% (laC-Eval) a 77,8% (este trabalho).

Discussão dos resultados

3 dimensões independentes

**Validade · Plan ·
Segurança**

O mesmo código pode ser válido, executável e ainda assim inseguro as três propriedades não se implicam.

+89,6pp

**Claude Opus 4 · L1
→ L3**

Especificidade de prompt eleva LLMs: +23,1pp também em Checkov no mesmo intervalo.

+43,7pp

GPT-5.4 · Trivy

Autocorreção via feedback do scanner funciona bem para Trivy, na maioria dos modelos.

02 · AVALIAÇÃO EXPERIMENTAL

Ameaças à validade

- **Poder estatístico baixo** em CodeLlama-13B e Magicoder-7B (n=14 e n=10).
- Cenários por consenso de LLMs podem **sobrerrepresentar padrões** dos dados de treinamento.
- Resultados refletem versões específicas dos modelos em maio de 2026, **sem seed fixa**.

03

Conclusões

O que aprendemos sobre segurança em IaC gerada por IA

Considerações finais

Contribuição

Primeiro
benchmark
dual-scanner:
**Checkov +
Trivy**, 7 modelos.

Evidência

WizardCoder-33
B: **77,8% válido,
0% seguro.**
Validade não
implica
segurança.

Impacto

Scanner
multi-ferramenta:
de complemento
a **pré-requisito.**

Trabalhos futuros

- Expandir para **outras linguagens de IaC**, provedores de nuvem e módulos oficiais como referência.
- Investigar **fine-tuning específico, RAG** e orientação por políticas para reduzir a lacuna sintaxe×segurança.
- Aumentar k por célula para fortalecer a **significância estatística** em modelos de baixa conformidade.
- Avaliar como artefatos das ferramentas podem ser **otimizados para Modelos de IA**.

Obrigado!

Perguntas são bem-vindas

AUTOR DE CONTATO

Francis Vargas

francisvargas.aluno@unipampa.edu.br

AI Horizon Labs · UNIPAMPA · PPGES



Slides reserva

Material de apoio para responder perguntas: detalhes de método, tabelas completas e provas.

Categorias de falha de validação

Por que os modelos falham

- Erros semânticos de dependência (Claude, GPT-5.4): referências a recursos corretas na intenção, ausentes na config.
- Versões de provider desatualizadas (Gemini, WizardCoder, Qwen): sintaxe AWS provider < v4, atributos depreciados.
- Erros estruturais de HCL (CodeLlama, Magicoder): blocos inválidos, markdown vazado, sintaxe Terraform 0.11/0.12.

Por que isso importa

- Falhas se concentram nos cenários mais sensíveis (KMS, VPC+CloudWatch) — nem penalizados, nem premiados.
- GPT-5.4: Resource cycle entre `aws_kms_key` e `aws_cloudwatch_log_group` — padrão seguro que o Terraform não instancia sem data source.
- Truncamento: CodeLlama-13B atinge o limite de contexto em 32,7% das gerações (4.096 tokens).

Concordância Checkov × Trivy (L3)

Modelo	Ambos	Só Trivy	Nenhum
Claude Opus 4	23,1%	69,4%	7,5%
Gemini 2.5 Pro	1,9%	77,4%	20,8%
GPT-5.4	1,4%	15,1%	83,6%
WizardCoder-33B	0,0%	0,8%	99,2%
Qwen2.5-Coder-14B	0,0%	6,4%	93,6%
CodeLlama-13B	4,3%	4,3%	91,3%
Magocoder-7B	0,0%	0,0%	100%